Step 1

什么是指针?

1.

语法：类型 *指针变量名;

指针变量的大小不是固定的，它取决于系统的寻址位数（即 CPU 架构和操作系统）。 在 32 位系统上，指针通常占 4 字节（32 位）。 在 64 位系统上，指针通常占 8 字节（64 位）。 指针的大小与所指向的数据类型无关。 例如，int *、char *、double * 在同一个平台上大小相同，因为它们都只是存储地址。

2.

20 10

int x = 10; x 初始值为 10。

int* p = &x; p 指向 x。

*p = 20; 通过 p 解引用，将 x 的值修改为 20。此时 x = 20。

int arr[3] = {3, 6, 9}; 数组 arr 初始化为 {3, 6, 9}，即 arr[0]=3, arr[1]=6, arr[2]=9。

int *q = arr; q 指向数组首元素 arr[0]。

int y = ++*arr + *++q; 这里有两个子表达式，按运算符优先级和结合性分析：

++*arr：*arr 是 arr[0]，值为 3。++ 是前置自增，所以 arr[0] 先自增为 4，整个表达式的值为自增后的值，即 4。

*++q：++q 是前置自增，q 原本指向 arr[0]，自增后指向 arr[1]。然后 * 解引用，取出 arr[1] 的值。此时 arr[1] 仍为 6（未被修改）。

因此 y = 4 + 6 = 10。

3.

野指针（Wild Pointer）是指指向不确定内存地址的指针。它可能指向随机位置、已释放的内存或无效地址。

常见来源：指针变量未初始化、指针指向的内存被释放（free）后未置空、指针越界访问。

危害 程序崩溃：访问非法地址导致段错误（Segmentation Fault）。

数据损坏：意外修改其他变量的值。

安全漏洞：攻击者可能利用野指针执行恶意代码。

难以调试：错误现象随机，难以定位。

如何避免 定义指针时立即初始化为 NULL。 使用 malloc 分配内存后检查是否为 NULL。 free 后立即将指针置为 NULL。 不要返回局部变量的地址。 使用前检查指针是否为 NULL。 避免指针越界。

4.

#include <stdio.h>

void swap(int *a, int *b) { int temp = *a; *a = *b; *b = temp; }

int main() { int x = 10, y = 20; printf("交换前: x = %d, y = %d\n", x, y); swap(&x, &y); // 传入地址 printf("交换后: x = %d, y = %d\n", x, y); return 0; }

什么是结构体?

1.

#include <stdio.h>

typedef struct PerInfo { char name[10]; char gender; int age; double height; } PerInfo;

2.

结构体指针是指向结构体变量的指针。它存储结构体变量的内存地址，可以通过指针间接访问结构体的成员。

3.

在 64 位系统（如 Windows/Linux 的 GCC/Clang）下，默认对齐规则通常为：

类型 大小（字节） 对齐要求（字节） char 1 1 int 4 4 double 8 8 数组 元素大小 × 长度 与元素类型的对齐要求相同

对齐规则：

每个成员相对于结构体起始地址的偏移量，必须是该成员对齐要求的整数倍。

结构体的总大小必须是其内部最大成员对齐要求的整数倍。

如果某个成员前有“空洞”，编译器会自动填充字节，使其满足对齐要求。

原始定义顺序：char name[10]; char gender; int age; double height;

成员 大小 对齐 起始偏移 占用范围
char name[10] 10 1 0 0 ~ 9 char gender 1 1 10 10 int age 4 4 12 12 ~ 15 double height 8 8 16 16 ~ 23 结构体总大小：当前结束于偏移 23，共 24 字节。最大对齐要求是 8（double），24 是 8 的整数倍，因此总大小为 24 字节。

Step 2

什么是链表?

1.

对比项 数组 链表 存储方式 在内存中连续存放，所有元素紧挨在一起 在内存中离散存放，每个节点通过指针连接 内存分配 通常编译时或运行时一次性分配固定大小 每个节点单独动态分配，按需增加，无需连续空间 访问

方式 支持随机访问，通过下标 arr[i] 直接计算地址 只能顺序访问，要从头节点开始逐个 next 查找 访问时间复杂度 O(1) O(n) 插入/删除 在中间插入或删除需要移动后续元素，平均 O(n) 修改指针即可，若已找到位置则 O(1) 大小 固定，扩容需要重新分配并复制 动态，可随时增加或减少节点 内存开销 仅存储数据本身，无额外指针开销 每个节点额外存储一个指针，内存开销更大 缓存友好性 连续存储，缓存命中率高，访问速度快 节点分散，缓存命中率低，访问速度相对较慢 越界风险 下标越界不会自动检查，可能导致未定义行为 只要指针有效，不会越界；但野指针或断链会导致问题

总结：数组适合频繁随机访问、元素数量固定的场景；链表适合频繁插入删除、元素数量动态变化的场景

2.

单向链表由若干节点组成，每个节点包含两部分：

数据域：存放实际数据，可以是整型、字符型、结构体等。

指针域：存放指向下一个节点的指针。最后一个节点的指针通常为 NULL，表示链表结束。

特点：

每个节点只知道自己的下一个节点，不能反向访问。

头节点（head）用于定位整个链表，通常需要单独保存。

插入、删除节点时，只需修改相邻节点的指针，不需要移动数据。

节点通常用 malloc 动态分配，用 free 释放。

#include <stdio.h> #include <stdlib.h>

// 定义单向链表节点，数据域为 int

typedef struct ListNode { int data; // 数据域：存储一个整数 struct ListNode *next; // 指针域：指向下一个节点 } ListNode;

实现基本的链操作

#include <stdio.h>

#include <stdlib.h>

#include <stdbool.h>

typedef struct Node {

```
int data;

struct Node *next;
```

} Node;

Node* creatnode(int data) {

```
    Node* newnode = (Node*)malloc(sizeof(Node));
    if (newnode == NULL) {
        return NULL;
    }
    newnode->data = data;
    newnode->next = NULL;
    return newnode;
```

}

Node* createhead() {

```
    Node* newnode = creatnode(0);
    return newnode;
```

}

void addfirst(Node **head, int data) {

```
    Node* newnode = creatnode(data);
    newnode->next = (*head)->next;
    (*head)->next = newnode;
```

}

void addtail(Node **head, int data) {

```
    Node* newnode = creatnode(data);
    Node* cur = *head;
    while (cur->next != NULL) {
        cur = cur->next;
    }
    cur->next = newnode;
```

}

void add(Node **head, int data, int pos) {

```
if (pos == 0) {

    addfirst(head, data);

}

else {

    Node* newnode = creatnode(data);
    Node* cur = *head;

    int i = 0;

    while (i < pos) {

        cur = cur->next;

        if (cur == NULL) {

            return;

        }

        i++;

    }
    newnode->next = cur->next;

    cur->next = newnode;

}
```

}

bool findnode(Node **head, int pos) {

```
Node* cur = *head;

int i = 0;

while (i < pos) {

    cur = cur->next;

    if (cur == NULL) {

        return false;
```

```
        }

        i++;

    }
    printf("pos=%d,data=%d", pos, cur->data);

    return true;
```

}

bool delfirst(Node **head) {

```
    Node* cur = *head;

    if (cur->next == NULL) {

        return false;

    }

    cur = cur->next;

    (*head)->next = cur->next;

    cur->next = NULL;

    free(cur);

    return true;
```

}

bool deltail(Node **head) {

```
    Node* cur = *head;

    if (cur->next == NULL) {

        return false;

    }

    while ((cur->next)->next != NULL) {

        cur = cur->next;

    }
```

```
    Node* target = cur->next;
    cur->next = NULL;

    free(target);

    return true;
```

}

bool del(Node **head, int pos) {

```
    Node* cur = *head;

    if (cur->next == NULL) {

        return false;

    }

    int i = 0;

    while (i < pos - 1) {

        cur = cur->next;

        if (cur == NULL) {

            return false;

        }

        i++;

    }

    Node* target = cur->next;

    cur->next = target->next;

    target->next = NULL;

    free(target);

    return true;
```

}

bool changenode(Node **head, int pos, int data) {

```
    Node* cur = *head;
    if (cur->next == NULL) {
        return false;
    }
    int i = 0;
    while (i < pos) {
        cur = cur->next;
        if (cur == NULL) {
            return false;
        }
        i++;
    }
    cur->data = data;
    return true;
```

}

void turnaround(Node **head) {

```
    if ((*head)->next == NULL) {
        return;
    }
    Node* first = (*head)->next;
    (*head)->next = NULL;
    Node* cur = first;
    Node* turn = *head;
    while (first->next != NULL) {
        while ((cur->next)->next != NULL) {
            cur = cur->next;
```

```
        }
        Node* target = cur->next;
        cur->next = NULL;
        turn->next = target;
        turn = turn->next;
        cur = first;
    }
    turn->next = first;
}
```